

Database Systems I / Information Systems I

SQL — Complementary Material

Christian Rauch

(changed: 2026-04-28)

Basic SELECT Structure

Every SQL query follows the same skeleton:

`SELECT columns FROM table(s) WHERE condition`

- ▶ **SELECT** – choose which columns (or expressions) to return
- ▶ **FROM** – name the table(s) to read from
- ▶ **WHERE** – filter rows *before* grouping

```
1  -- Names of artists born before 1900
2  SELECT name
3  FROM   Artists
4  WHERE  birthDate < '1900-01-01';
```

Filtering with WHERE

- ▶ Comparison operators: =, <>, <, >, <=, >=
- ▶ Logical operators: AND, OR, NOT
- ▶ Set membership: IN (...), NOT IN (...)
- ▶ Pattern matching: LIKE '%pattern%'
- ▶ NULL test: IS NULL, IS NOT NULL

```
1  -- Artifacts worth more than 100,000
2  SELECT id, title, value
3  FROM    Artifacts
4  WHERE   value > 100000;
5
6  -- Artists whose name contains 'Monet'
7  SELECT id, name
8  FROM    Artists
9  WHERE   name LIKE '%Monet%';
```

Aggregate Functions

Aggregate functions collapse many rows into a single value.

Function	Meaning
COUNT(*)	number of rows
COUNT(DISTINCT col)	distinct non-NULL values
SUM(col)	total
AVG(col)	arithmetic mean
MIN(col) / MAX(col)	extremes

```
1  -- Total and average artifact value per
   collection
2  SELECT    collectionTitle,
3            COUNT(*)      AS numArtifacts,
4            SUM(value)    AS totalValue,
5            AVG(value)    AS avgValue
6  FROM      Artifacts
7  WHERE     collectionTitle IS NOT NULL
8  GROUP BY  collectionTitle;
```

GROUP BY and HAVING

Rule

Every column in SELECT must either be in GROUP BY or wrapped in an aggregate function.

- ▶ **GROUP BY** – partition rows into groups
- ▶ **HAVING** – filter *groups* (applied after aggregation)

```
1  -- Artists with more than one artifact in the
   database
2  SELECT  artistId, COUNT(*) AS numArtifacts
3  FROM    Artifacts
4  GROUP BY artistId
5  HAVING  COUNT(*) > 1;
6
7  -- Total advertisement duration per medium
8  SELECT  mediumName, SUM(duration) AS totalDays
9  FROM    Advertisements
10 GROUP BY mediumName;
```

Join Types

Type	Returns
(INNER) JOIN	only rows with a match in <i>both</i> tables
LEFT JOIN	all rows from the left table; NULLs for unmatched right rows
RIGHT JOIN	all rows from the right table; NULLs for unmatched left rows
FULL JOIN	all rows from both tables

```
1  -- Artifact titles together with their artist's  
   name  
2  SELECT a.title, ar.name AS artistName  
3  FROM   Artifacts a  
4  JOIN   Artists ar ON a.artistId = ar.id;
```

Self-Join

A table can be joined with *itself* using two aliases. Useful for finding pairs within the same set.

```
1  -- Pairs of artists born in the same year
2  SELECT ar1.name AS artist1,
3         ar2.name AS artist2,
4         YEAR(ar1.birthDate) AS birthYear
5  FROM   Artists ar1
6  JOIN   Artists ar2
7         ON  YEAR(ar1.birthDate) = YEAR(ar2.
8         birthDate)
9         AND ar1.id < ar2.id;    -- avoids (A,B) and
                                (B,A)
```

The condition `ar1.id < ar2.id` removes both duplicates and self-pairs in one step.

Subqueries

A *subquery* is a SELECT nested inside another statement.

Scalar returns a single value; used with =, >, ...

Row/Table returns a set; used with IN, NOT IN, EXISTS

Derived table used in FROM with an alias

```
1  -- Scalar subquery: most expensive artifact
2  SELECT title, value
3  FROM    Artifacts
4  WHERE   value = (SELECT MAX(value) FROM Artifacts
5              );
6
7  -- IN subquery: artifacts in a specific
8      collection topic
9  SELECT id, title
10 FROM    Artifacts
11 WHERE   collectionTitle IN (
12         SELECT title FROM Collections WHERE topic =
13             'painting'
14     );
```


Correlated vs. Uncorrelated Subqueries

Correlated

References a column from the outer query – re-executed for every outer row.

Uncorrelated

Independent of the outer query – executed *once*; result reused.

Both styles produce the same result. The uncorrelated version (derived table or JOIN) is usually more efficient because the subquery runs only once.

The next three slides show the same query written both ways.

Example 1: Scalar Comparison (Correlated)

Task: Find artifacts whose value exceeds the average value of all artifacts by the same artist.

Correlated – subquery references outer alias a, so it is re-evaluated once per artifact:

```
1 SELECT id, title, value, artistId
2 FROM   Artifacts a
3 WHERE  value > (
4         SELECT AVG(value)
5         FROM   Artifacts b
6         WHERE  b.artistId = a.artistId  --
7         references outer row
8     );
```

Example 1: Scalar Comparison (Uncorrelated)

Task: same – now rewritten without correlation.

Uncorrelated – a derived table computes all averages *once*, then the outer query joins against it:

```
1 SELECT a.id, a.title, a.value, a.artistId
2 FROM   Artifacts a
3 JOIN   (SELECT artistId, AVG(value) AS avgVal
4         FROM   Artifacts
5         GROUP BY artistId) avg_a
6       ON a.artistId = avg_a.artistId
7 WHERE  a.value > avg_a.avgVal;
```

Example 2: EXISTS (Correlated)

Task: Find media outlets that have placed at least one advertisement.

Correlated – EXISTS references the outer alias `m`, so the subquery is re-evaluated for every medium:

```
1 SELECT name
2 FROM   Medium m
3 WHERE  EXISTS (
4         SELECT 1
5         FROM    Advertisements a
6         WHERE   a.mediumName = m.name  -- references
           outer row
7 );
```

Example 2: IN (Uncorrelated)

Task: same – now rewritten without correlation.

Uncorrelated – the inner SELECT is fully independent; it runs once and produces a set for IN:

```
1 SELECT name
2 FROM   Medium
3 WHERE  name IN (
4         SELECT DISTINCT mediumName    -- independent
5         FROM   Advertisements
6     );
```

Example 3: NOT EXISTS (Correlated)

Task: Find artists who have no artifact in the database.

Correlated – NOT EXISTS references the outer alias ar, re-evaluated for every artist:

```
1 SELECT id, name
2 FROM   Artists ar
3 WHERE  NOT EXISTS (
4         SELECT 1
5         FROM   Artifacts a
6         WHERE  a.artistId = ar.id  -- references
           outer row
7 );
```

Example 3: NOT IN (Uncorrelated)

Task: same – now rewritten without correlation.

Uncorrelated – the subquery builds the full set of artist IDs once; NOT IN filters against it:

```
1 SELECT id, name
2 FROM Artists
3 WHERE id NOT IN (
4     SELECT DISTINCT artistId      -- independent
5     FROM Artifacts
6     WHERE artistId IS NOT NULL
7 );
```

Note: the IS NOT NULL guard is necessary – NOT IN returns no rows at all if the subquery contains any NULL value.

EXISTS

EXISTS returns TRUE if the subquery returns at least one row.
Naturally correlated; stops as soon as one match is found.

```
1  -- Collections that contain at least one
   painting
2  SELECT title
3  FROM    Collections c
4  WHERE   EXISTS (
5          SELECT 1
6          FROM    Artifacts a
7          JOIN    ArtifactPaintings ap ON a.id = ap.id
8          WHERE   a.collectionTitle = c.title
9  );
```

SELECT 1 (or SELECT *) is conventional inside EXISTS – the actual column values are irrelevant.

INSERT

```
1  -- Single row
2  INSERT INTO Artists (id, name, birthDate,
3     deathDate, bio)
4     VALUES (6, 'Pablo Picasso', '1881-10-25', '
5     1973-04-08', NULL);
6
7  -- Insert from a SELECT (copy a subset of rows)
8  INSERT INTO ExhibitedAt (artifactId,
9     exhibitionTitle)
10 SELECT a.id, 'Modern Visions'
11 FROM   Artifacts a
12 JOIN   ArtifactSculptures s ON a.id = s.id
13 WHERE  a.collectionTitle = 'Modern Art';
```

The INSERT ...SELECT pattern copies data without leaving the database.

UPDATE

```
1  -- Set a placeholder description for artifacts  
   with none  
2  UPDATE Artifacts  
3  SET    description = 'No description available'  
4  WHERE  description IS NULL;  
5  
6  -- Move all exhibitions from room '01-100' to  
   '01-150'  
7  UPDATE Exhibitions  
8  SET    room = '01-150'  
9  WHERE  room = '01-100';
```

DELETE

```
1  -- Remove advertisements with zero duration
2  DELETE FROM Advertisements
3  WHERE duration = 0;
```

Caution

DELETE without a WHERE clause removes *all* rows.

Always test with SELECT first.

Handling NULL

- ▶ NULL means *unknown* – not zero, not empty string.
- ▶ Any arithmetic or comparison with NULL yields NULL.
- ▶ Test with IS NULL / IS NOT NULL, not with = NULL.
- ▶ COUNT(*) counts all rows; COUNT(col) ignores NULLs.
- ▶ LEFT JOIN produces NULLs for unmatched rows – useful for "find rows with no match" patterns.

```
1  -- Artifacts not yet assigned to any collection
2  SELECT id, title
3  FROM    Artifacts
4  WHERE   collectionTitle IS NULL;
5
6  -- Artists with no biographical information
7  SELECT id, name FROM Artists WHERE bio IS NULL;
```

ALTER TABLE – Adding and Removing Columns

```
1  -- Add a new nullable column
2  ALTER TABLE Artifacts
3      ADD COLUMN location VARCHAR(100);
4
5  -- Add a column with a default value
6  ALTER TABLE Exhibitions
7      ADD COLUMN capacity INT DEFAULT 0;
8
9  -- Drop a column
10 ALTER TABLE Artists
11     DROP COLUMN bio;
```

Caution

DROP COLUMN permanently destroys the data in that column.

ALTER TABLE – Modifying Columns

```
1  -- Change the data type of a column
2  ALTER TABLE Artifacts
3      MODIFY COLUMN year INT;
4
5  -- Rename a column (and optionally change its
6      type)
7  ALTER TABLE Artists
8      CHANGE COLUMN name fullName VARCHAR(100);
9
10 -- Give a column a new default value
11 ALTER TABLE Exhibitions
12     ALTER COLUMN room SET DEFAULT '01-100';
13
14 -- Remove a default value
15 ALTER TABLE Exhibitions
16     ALTER COLUMN room DROP DEFAULT;
```

ALTER TABLE – Constraints

```
1  -- Add a FOREIGN KEY constraint
2  ALTER TABLE Artifacts
3      ADD CONSTRAINT fk_artist
4      FOREIGN KEY (artistId) REFERENCES Artists(id);
5
6  -- Add a UNIQUE constraint
7  ALTER TABLE Medium
8      ADD CONSTRAINT uq_contact UNIQUE (contact);
9
10 -- Drop a constraint by name
11 ALTER TABLE Artifacts
12     DROP FOREIGN KEY fk_artist;
13
14 -- Rename the table itself
15 ALTER TABLE ArtifactPaintings
16     RENAME TO Paintings;
```

Date Functions

Function	Returns
CURDATE()	today's date
NOW()	date + time
YEAR(d)	year part
MONTH(d)	month part
DAY(d)	day part
DATEDIFF(a,b)	days $a - b$

```
1  -- Exhibitions starting
2  -- in the current year
3  SELECT title, startDate
4  FROM    Exhibitions
5  WHERE   YEAR(startDate)
6          = YEAR(CURDATE());
7
8  -- Days running so far
9  SELECT title,
10         DATEDIFF(CURDATE(),
11                 startDate)
12         AS daysRunning
13  FROM    Exhibitions
14  WHERE   startDate <=
          CURDATE();
```


String Functions

Function	Effect
UPPER(s) / LOWER(s)	change case
LENGTH(s)	number of characters
TRIM(s)	remove leading/trailing spaces
SUBSTRING(s, pos, len)	extract substring
CONCAT(s1, s2, ...)	concatenate strings
REPLACE(s, old, new)	substitute substring

```
1  -- Artists whose name has more than 15
   characters
2  SELECT name, LENGTH(name) AS nameLen
3  FROM   Artists
4  WHERE  LENGTH(name) > 15;
5
6  -- Display collection titles in upper case
7  SELECT UPPER(title) AS titleUpper
8  FROM   Collections;
```

Pattern Matching with LIKE

LIKE matches strings using two wildcards:

Wildcard	Matches	Example
%	any sequence of characters (incl. empty)	'Van%'
_	exactly one character	'_onet'

```
1  -- Names starting with 'Van'
2  SELECT name FROM Artists WHERE name LIKE 'Van%';
3
4  -- Names where the second character is 'o' (e.g.
   'Monet', 'Rodin')
5  SELECT name FROM Artists WHERE name LIKE '_o%';
6
7  -- Titles containing the word 'Night' anywhere
8  SELECT title FROM Artifacts WHERE title LIKE '%
   Night%';
9
10 -- Titles that are exactly 10 characters long
11 SELECT title FROM Artifacts WHERE title LIKE '
   , .
```